# Sprint 6 · Reto «Datos sucios, cliente impaciente»

**Ficha completa del reto, para consultar.** Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

## RETO Ficha del reto

### 1 Qué os pide el cliente

**Qué os pide el cliente.** TurisData Canarias os manda por correo un fichero de reservas, `reservas_sucio.csv`, **exportado a mano desde varios sistemas**. El gerente quiere un primer modelo que prediga qué reservas se cancelan «para ayer», pero **no se fía de los datos** (con razón). Os pide dos cosas: un **informe corto** que diga **qué está mal en los datos y por qué**, y un **proceso automático** (una función y un *pipeline*, es decir, una cadena de pasos que se ejecuta siempre igual) que deje los datos listos cada vez que le llegue un fichero así, con un primer modelo sencillo de referencia (un *baseline*) para saber si merece la pena seguir.

**Cómo se trabaja.** Los datos reales siempre vienen sucios (fechas escritas de varias maneras, países con distintos nombres, números imposibles, filas repetidas, huecos…) y limpiarlos es entre el 60 y el 80 % del trabajo en un proyecto de IA. No hace falta experiencia previa: ya habéis practicado cada técnica en los cuadernos del sprint, y aquí las juntáis en un caso real. Regla de oro: **toda corrección va escrita en código**, nunca editando el CSV a mano, para que se pueda repetir. En clase tenéis 10 minutos de planning (día 1), **80 minutos de reto** (30 el día 1 y 50 el día 2) y **35 de entrega, review y retrospectiva** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

## 2 · Recursos del reto

| Recurso | Para qué lo usaréis | Dónde está |
|---|---|---|
| **Google Colab** (Python con `pandas`, `matplotlib`, `seaborn` y `scikit-learn`, ya instalados) | Escribir y ejecutar `reto_S06.ipynb` | Colab, con vuestra cuenta de Google. Solo librerías del curso |
| **El CSV sucio** `reservas_sucio.csv` | Los datos del cliente (sintéticos, sin datos personales). **No se edita a mano** | Los cuadernos del sprint lo descargan solos con la función `dato(...)` de su primera celda; cópiala en vuestro cuaderno |
| **La plantilla del informe** `informe_calidad_plantilla.md` | Tabla de defectos, antes y después, preguntas al cliente, IA responsable y revisión con la rúbrica | En GitHub: abrid el archivo y copiad el texto con el botón *Raw* |
| **Cuadernos del sprint**: `S06_01` (EDA y `resumen_calidad`) y `S06_02` (limpieza y mini-reto `limpiar_reservas`) | Las técnicas y el código de partida para el inventario y la limpieza | Colab; su código explicado |
| **Cuadernos del sprint**: `S06_03` (*pipeline*) y `S06_04` (*baseline*, métricas y validación cruzada) | El código de partida para el *pipeline* y la comparación con el modelo «tonto» | Colab; su código explicado: `S06_03` y `S06_04` |

| Recurso | Para qué lo usaréis | Dónde está |
|---|---|---|
| **Cómo entregar en GitHub** | Subir la carpeta `sprint-06/` (incluido el cuaderno de Colab, apartado 6 de la guía) | Guía de entrega |
| **Autoevaluación y coevaluación** | Cada persona rellena su hoja | Hoja del sprint |
| **Un asistente de IA** (opcional) | Entender errores y pedir pistas | Con las reglas de la «Ayuda 1» del paso a paso |

## 3 · Guía paso a paso

Son cinco pasos, de **90 minutos en total** (10 de planning y 80 de reto). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

## Palabras que van a aparecer en este reto

| Palabra | Qué significa, en sencillo |
|---|---|
| **Defecto de calidad** | Un problema del dato: huecos (nulos), duplicados, valores imposibles, formatos mezclados… |
| **Imputar** | Rellenar un hueco con un valor razonable (la mediana, la categoría más frecuente…) |
| **Outlier** (valor atípico) | Un valor muy raro. A veces es un error y a veces es real: hay que decidir |
| **Train / test** | Los datos con los que entrena el modelo (*train*) y los que se guardan para evaluarlo (*test*) |
| **Pipeline** | Una cadena de pasos de preparación y modelo que se ejecuta siempre igual |
| **Fuga de datos** | Cuando información del *test* «se cuela» en el entrenamiento y los resultados parecen mejores de lo que son |
| **Baseline y `DummyClassifier`** | El modelo de referencia, y un «modelo tonto» (por ejemplo, responde siempre la clase más frecuente) que es el suelo con el que comparar |
| **Validación cruzada** | Entrenar y evaluar varias veces con particiones distintas para tener una medida más fiable (media ± desviación) |

## Paso 1 · Planning y reparto de roles (día 1, 10 minutos)

1. Elegir **Scrum Master y responsable técnico** de este sprint (no pueden repetir quienes lo fueron en el anterior).
2. Leer los apartados 1 y 3 de esta ficha y decidir hasta qué nivel queréis llegar.
3. Pasar al tablero las tareas y decidir quién empieza cada una.

**En sencillo.** Este paso no tiene código: es una reunión corta de equipo, como la de varias personas que van a cocinar juntas y, antes de encender el fuego, deciden quién lleva el tiempo, quién comprueba que la receta saldría igual otro día y quién empieza por qué plato. El cliente (TurisData) os ha mandado un fichero de reservas «sucio»; en los pasos siguientes lo vais a revisar, limpiar y probar con un modelo sencillo. Aquí solo organizáis el trabajo para no pisaros: dos roles, un nivel de ambición y un tablero con las tareas a la vista. No hace falta saber Python ni Scrum: con lo que pone esta página basta, y el reparto se puede cambiar más adelante si no funciona.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Planning** | La reunión de arranque del sprint: se lee el reto, se reparte el trabajo y se apuntan las tareas. Aquí dura 10 minutos. |

| Palabra | Qué significa, en sencillo |
|---|---|
| **Scrum Master** | Quien cuida que el equipo trabaje bien: lleva el tiempo, mantiene el tablero al día y avisa si algo se atasca. No es el jefe ni quien hace más trabajo. |
| **Responsable técnico** | Quien vigila que el resultado sea correcto y se pueda repetir: que otra persona ejecute el cuaderno y obtenga lo mismo. No lo programa todo. |
| **Tablero y tarjeta** | El panel del equipo (Trello o similar) con columnas; cada tarea es una tarjeta con una persona responsable. Si no está en el tablero, para el equipo no existe. |
| **Nivel** | Hasta dónde llega el equipo en este reto: mínimo (una entrega válida), estándar o extra. |
| **Inventario de defectos** | La lista de lo que está mal en el fichero: huecos, duplicados, valores imposibles, formatos mezclados… (paso 2). |
| **Función de limpieza** | Un trozo de código con nombre que arregla el fichero siempre igual, sin tocar el original (paso 3). |
| **Pipeline** | Una cadena de pasos de preparación y modelo que se ejecuta siempre igual (paso 4). |
| **Baseline** | El modelo de referencia, muy simple: el «suelo» con el que comparar el vuestro (paso 4). |

### Cómo se hace

1. **Elegir los dos roles (3 minutos).** Abrid la tarjeta *Roles* del tablero (columna *Información*) y elegid **Scrum Master** y **responsable técnico** para el Sprint 6. Reglas de la guía del tablero: las dos personas no pueden ser las mismas que en el sprint anterior y nadie repite hasta que hayan pasado todas las personas del equipo. Apuntad los dos nombres en la fila del Sprint 6. Los roles solo reparten **quién vigila qué**: todo el equipo trabaja en el contenido.

2. **Decidir el nivel (3 minutos).** Leed el apartado 1 («Qué os pide el cliente») y el 3 («Guía paso a paso») de la ficha del reto; los cinco pasos resumidos están en el reto en una página. Después elegid hasta dónde llegáis:

| Nivel | Qué hay que tener |
|---|---|
| **Mínimo** | Tabla de **6 defectos o más** · una función `limpiar_reservas` que devuelve la tabla limpia y un resumen · el cuaderno corre con *Reiniciar y ejecutar todo* · README corto · todo subido a `sprint-06/`. Es una entrega válida y evaluable. |
| **Estándar** | Además: un *pipeline* que ajusta solo con *train* · el modelo y el «modelo tonto» comparados con validación cruzada · 3 preguntas al cliente · IA responsable · README con versiones · revisión con la rúbrica. |
| **Extra** | Además: 8–10 defectos · partición temporal justificada · explicar o probar qué pasa si se escala con todos los datos. |

   Si dudáis, apuntad el **mínimo** y subid de nivel si os sobra tiempo: si el tiempo aprieta, se completa el mínimo y lo demás se deja para casa. Anotad el nivel en el tablero, donde todo el equipo lo vea.

3. **Crear las seis tarjetas (3 minutos).** En la columna *Sprint backlog* cread una tarjeta por tarea, con **una persona responsable** y, en la descripción, cuándo se da por hecha:

| Tarjeta | Se da por hecha cuando… |
|---|---|
| Inventario de defectos | Hay 6 o más defectos con sus datos completos (paso 2). |
| Función de limpieza | `limpiar_reservas` devuelve tabla y resumen sin tocar el original, y *train* y *test* están separados (paso 3). |
| *Pipeline* | El preprocesado imputa, escala y codifica ajustando solo con *train* (paso 4). |
| *Baseline* | Hay una frase con media ± desviación, modelo frente a modelo «tonto» (paso 4). |
| Informe | Todos los apartados del nivel elegido están rellenos (paso 5). |
| README | Otra persona podría repetir el cuaderno con solo el README (paso 5). |

   Quien empieza una tarjeta la mueve a *En curso* (máximo 2 por persona). Al terminar, a *En revisión*; la pasa a *Hecho* otra persona: nadie revisa lo suyo.

4. **Repartir quién empieza (1 minuto).** Todo el equipo debe entender todo, así que **quien no programa una parte, la revisa**. Un reparto posible para los pasos 2 a 4:

| Persona | Empieza por | Revisa |
|---|---|---|
| A | Inventario de defectos y tabla del informe | La función de limpieza |
| B | Función `limpiar_reservas` y partición *train*/*test* | El *pipeline* |
| C | *Pipeline*, *baseline* y validación cruzada | El inventario de defectos |
| D (si hay 4) | README y preguntas al cliente | El informe entero |

   Si sois 3, repartid lo de D entre las otras tres personas. Para cerrar, cada persona dice en voz alta: «Empiezo por… y reviso…».

**Qué deberíais ver.** En el tablero, seis tarjetas en *Sprint backlog* (las de quien ya empieza, en *En curso*), cada una con su persona responsable y su criterio de «hecha»; los nombres del Scrum Master y del responsable técnico en la tarjeta *Roles*; y el nivel anotado. Por ejemplo:

```
 Sprint backlog                       Persona
   Inventario de defectos ..... A
   Función de limpieza ..... B
   Pipeline ..... C
   Baseline ..... C
   Informe ..... A
   README ..... D
Nivel del equipo: mínimo (subimos si sobra tiempo)
```

**Hecho cuando:** los roles están elegidos y cada persona sabe con qué tarea empieza y qué parte revisa.

### Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| No os ponéis de acuerdo en el nivel | Cada persona tiene una idea distinta de cuánto da tiempo. | Apuntad el **mínimo** (es una entrega válida) y decidid en la *daily* si subís; el estándar es lo que describe la «Definition of Done» de la ficha completa. |
| Sois 3 y el reparto tiene 4 filas | La fila D es «si hay 4». | Repartid el README y las preguntas al cliente entre A, B y C; el informe entero lo revisa quien no lo escribió. |
| Alguien repite rol respecto al Sprint 5 | Los dos roles cambian en cada sprint. | Elegid a otras personas y anotad el relevo en la tarjeta *Roles*. |
| Todas las tarjetas acaban en la misma persona | Se confunde «rol» con «hacerlo todo». | El Scrum Master facilita, no hace todo: repartid con la tabla de arriba y no pongáis más de 2 tarjetas *En curso* por persona. |
| No tenéis tablero | No se montó en el Sprint 0 o se perdió. | Montadlo con la guía del tablero; basta con las columnas *Sprint backlog*, *En curso*, *En revisión* y *Hecho*. |
| Pasan los 10 minutos y falta algo | El planning es corto a propósito. | Apuntad lo que falta como tarjeta y empezad; el reparto se ajusta en la *daily*. |

## Paso 2 · Inventario de defectos (día 1, 30 minutos)

1. Abrir un cuaderno nuevo en Colab (`reto_S06`), cargar `reservas_sucio.csv` y pasar `resumen_calidad`.
2. Buscar defectos con tablas y gráficos, categoría por categoría.
3. Rellenar **una fila de la tabla del informe por cada defecto** (mínimo 6).
4. Apuntar las preguntas que haríais al cliente.

**En sencillo.** Es como revisar un coche usado antes de comprarlo: no se arregla nada todavía, solo se mira con una lista y se anota cada fallo (qué es, dónde está, cuántos hay y cómo lo habéis visto). El «coche» es `reservas_sucio.csv`, un fichero de reservas que el cliente exportó a mano desde varios sistemas. Vais a escribir unas pocas celdas de código que **cuentan y muestran** cosas, y con lo que salga rellenaréis una tabla. Nada de lo que escribáis cambia el fichero: solo lo leéis. No hace falta saber programar: copiáis las celdas tal cual y cambiáis, como mucho, el nombre de una columna. Lo difícil no es el código, es **fijarse** en lo que sale.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Defecto de calidad** | Algo del fichero que no cuadra con la realidad: un hueco, una repetición, un formato mezclado, un valor imposible. |
| `crudo` | El nombre de la tabla tal y como llega del cliente. No se modifica nunca: sirve para comparar al final. |
| **Función (`def`)** | Un trozo de código con nombre que se escribe una vez y se usa muchas. Aquí, `dato` y `resumen_calidad`. |
| **Nulo (`NaN`)** | Una casilla vacía, sin dato. |

| Palabra | Qué significa, en sencillo |
|---|---|
| **Tipo de dato** | Cómo guarda pandas una columna: `int64` (entero), `float64` (decimal), `object` (texto). Un precio en `object` es sospechoso. |
| **Duplicado** | Una fila idéntica a otra en todas sus columnas. |
| **Categoría inconsistente** | Lo mismo escrito de varias formas: `directo`, `Directo` y `DIRECTO` son para el ordenador tres cosas distintas. |
| **Valor imposible** | Un dato que no puede ser verdad (un precio negativo). Un *outlier* es un valor extremo pero **posible**: ante la duda, es una pregunta para el cliente. |
| `errors="coerce"` | «Si no sabes convertir esto, ponlo como nulo en vez de dar error». Sirve para probar un formato de fecha sin que el cuaderno se pare. |

### Cómo se hace

1. **Abrir el cuaderno.** En Colab, *Archivo* → *Nuevo cuaderno* y renombradlo `reto_S06`. Con *Compartir* (arriba a la derecha) dad acceso de editor al resto del equipo. Una persona escribe y el resto mira y propone; cambiad de turno cada pocos minutos. Cada bloque de código de abajo va en **una celda nueva** (*+ Código*), y se ejecuta con el botón ▶ o con Mayús + Intro. Ejecutad **de arriba abajo**: cada celda usa nombres que crean las anteriores.
2. **Preparar y cargar el CSV.** Primera celda:

```
import os, urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

BASE_URL = "https://raw.githubusercontent.com/cCoolBoox/IFCD107-alumnado/main/datos/"

def dato(nombre):
    """Devuelve la ruta local de un archivo de datos del curso (lo descarga si no está)."""
    for carpeta in ("datos", "../datos", "../../datos"):
        ruta = os.path.join(carpeta, nombre)
        if os.path.exists(ruta):
            return ruta
    os.makedirs("datos", exist_ok=True)
    ruta = os.path.join("datos", nombre)
    urllib.request.urlretrieve(BASE_URL + nombre, ruta)
    return ruta

SEMILLA = 42
sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", 30)

crudo = pd.read_csv(dato("reservas_sucio.csv"))
print(crudo.shape)
```

   Qué hace cada línea:

   - `import … as pd`: trae las herramientas (pandas para tablas, matplotlib y seaborn para gráficos). Los nombres cortos `pd`, `plt`, `sns` y `np` son los de siempre: no los cambiéis.
   - `def dato(nombre):`: define la función `dato`. Busca el fichero en tres carpetas y, si no está, lo descarga de GitHub a la carpeta `datos`. Es la misma que usan los cuadernos `S06_01` y `S06_02`.
   - `SEMILLA = 42`: el número que fija el azar para que todo salga igual en cada ejecución.
   - `pd.set_option(…)`: pide que las tablas anchas muestren hasta 30 columnas.
   - `crudo = pd.read_csv(dato("reservas_sucio.csv"))`: lee el fichero y lo guarda en `crudo`. **Esta tabla no se modifica nunca.**
   - `print(crudo.shape)`: imprime (filas, columnas).

3. **Mirar el estado de cada columna.** Celda nueva:

```
def resumen_calidad(tabla):
    return pd.DataFrame({
        "tipo": tabla.dtypes.astype(str),
        "nulos": tabla.isna().sum(),
        "pct_nulos": (tabla.isna().mean() * 100).round(2),
        "distintos": tabla.nunique(),
    })

resumen_calidad(crudo)
```

   Qué hace cada línea:

   - `def resumen_calidad(tabla):` : una función que recibe una tabla y devuelve otra con una fila por columna.
   - `"tipo"`, `"nulos"`, `"pct_nulos"`, `"distintos"` : el tipo de dato, cuántas casillas vacías, el porcentaje de vacías y cuántos valores diferentes hay.
   - `resumen_calidad(crudo)` : al ser la **última línea** de la celda, Colab muestra su resultado.

   Después, en otra celda, `crudo.describe().round(1)` da mínimo, máximo y media de las columnas numéricas.

4. **Duplicados, tipos y fechas.** Celda nueva:

```
print("Filas duplicadas exactas:", crudo.duplicated().sum())
print("Precios escritos con coma:", crudo["precio_noche"].astype(str).str.contains(",").sum())
iso = pd.to_datetime(crudo["fecha_reserva"], format="%Y-%m-%d", errors="coerce")
espanol = pd.to_datetime(crudo["fecha_reserva"], format="%d/%m/%Y", errors="coerce")
print("Fechas ISO:", iso.notna().sum(), "· dd/mm/aaaa:", espanol.notna().sum(), "· sin leer:", (iso.isna() &
espanol.isna()).sum())
crudo[["fecha_reserva", "precio_noche"]].head(8)
```

   Qué hace cada línea:

   - `crudo.duplicated().sum()` : marca con «verdadero» cada fila repetida y suma cuántas hay.
   - `.astype(str).str.contains(",")` : convierte la columna a texto y marca los precios que llevan una coma (por ejemplo `83,36`).
   - `iso = …` y `espanol = …` : intentan leer la fecha en dos formatos. `%Y-%m-%d` es `2025-11-09` y `%d/%m/%Y` es `19/11/2024`. Donde no encaja, `errors="coerce"` deja un nulo.
   - `iso.notna().sum()` : cuenta las fechas que se han leído con ese formato; `(iso.isna() & espanol.isna()).sum()` cuenta las que no se leen con ninguno.
   - `.head(8)` : enseña las 8 primeras filas de esas dos columnas para verlas con vuestros ojos.

5. **Categorías inconsistentes.** Celda nueva. Cambiad el nombre de la columna para revisar las demás de texto (`tipo_habitacion`, `regimen`, `tarifa`, `deposito`):

```
print(crudo["canal"].value_counts().to_string())
print()
print(crudo["pais_origen"].value_counts().to_string())
```

   Qué hace cada línea:

   - `crudo["canal"]` : elige la columna `canal` (el nombre va entre comillas y debe ser exacto).
   - `.value_counts()` : cuenta cuántas veces aparece cada valor distinto.
   - `.to_string()` : lo escribe completo, sin recortar filas.
   - `print()` solo deja una línea en blanco entre las dos listas.

6. **Valores imposibles y *outliers*.** Celda nueva:

```
precio = pd.to_numeric(crudo["precio_noche"].astype(str).str.replace(",", "."), errors="coerce")
print(precio.describe().round(1))
print("Precios <= 0:", (precio <= 0).sum(), "· precios > 1000:", (precio > 1000).sum())
print("Noches <= 0:", (crudo["noches"] <= 0).sum())
print("Antelaciones negativas:", (crudo["antelacion_dias"] < 0).sum())
```

   Qué hace cada línea:

   - `precio = …`: copia la columna de precios cambiando la coma por un punto y la convierte en número. Es una copia aparte: `crudo` no cambia.
   - `precio.describe()`: mínimo, máximo, media… del precio.
   - `(precio <= 0).sum()`: cuenta los precios que cumplen la condición. Cambiad el `0` o el `1000` si queréis probar otros límites.
   - `noches` y `antelacion_dias`: el mismo truco con otras dos columnas que no pueden ser negativas.

   Para ver los *outliers*, un diagrama de caja con los precios razonables (la caja es donde está la mayoría; los puntos sueltos son los extremos):

```
precio_ok = precio[precio.between(1, 1000)]
fig, ejes = plt.subplots(1, 2, figsize=(11, 3.5))
sns.boxplot(x=precio_ok, ax=ejes[0], color="#8ecae6")
ejes[0].set(title="Precio de todas las reservas", xlabel="Precio por noche (€)")
sns.boxplot(x=precio_ok, y=crudo.loc[precio_ok.index, "tipo_habitacion"], ax=ejes[1], color="#8ecae6")
ejes[1].set(title="Precio por tipo de habitación", xlabel="Precio por noche (€)", ylabel="Tipo de habitación")
plt.tight_layout()
plt.show()
```

   - `precio.between(1, 1000)`: se queda con los precios entre 1 y 1000 para que los 9999 no aplasten el dibujo.
   - `plt.subplots(1, 2, …)`: prepara dos gráficos lado a lado; `sns.boxplot(…, ax=ejes[0])` dibuja uno en cada hueco.
   - `.set(title=…, xlabel=…)`: pone título y nombres a los ejes. `plt.show()` dibuja el resultado.

7. **Nulos e incoherencias entre columnas.** Celda nueva:

```
nulos = crudo.isna().sum()
print(nulos[nulos > 0])
fecha_reserva = iso.fillna(espanol)
fecha_llegada = pd.to_datetime(crudo["fecha_llegada"], format="%Y-%m-%d")
antelacion_calculada = (fecha_llegada - fecha_reserva).dt.days
print("Antelaciones que no cuadran con las fechas:", (antelacion_calculada != crudo["antelacion_dias"]).sum())
```

   Qué hace cada línea:

   - `nulos[nulos > 0]`: de todas las columnas, deja solo las que tienen algún hueco. **Mirad todas**, también las de texto como `regimen`.
   - `iso.fillna(espanol)`: rellena las fechas que `iso` no pudo leer con las que sí leyó `espanol`; así hay una fecha de reserva para cada fila.
   - `(fecha_llegada - fecha_reserva).dt.days`: los días entre reservar y llegar, calculados por vosotros.
   - `!= crudo["antelacion_dias"]`: compara con lo que dice el fichero y cuenta las filas donde no coincide.

8. **Rellenar la tabla de defectos.** Una persona crea en el repositorio del equipo, dentro de `sprint-06/`, el archivo `informe_calidad.md`: abre `informe_calidad_plantilla.md`, pulsa *Raw*, copia todo el texto, y en GitHub usa *Add file* → *Create new file*. Por **cada defecto**, una fila de la tabla del apartado 2, con ocho columnas. Un ejemplo con los números reales de este fichero:

| Columna de la plantilla | Qué se escribe |
|---|---|
| N° | 1 |
| Defecto | Precios escritos con coma decimal ( `83,36` ) |
| Columna(s) | `precio_noche` |
| Filas afectadas | 153 de 1545 |
| Cómo lo detectamos | `resumen_calidad` dice `object` (texto) en una columna de precios; `.str.contains(",")` los cuenta |
| Causa probable | Exportación desde un Excel en español |
| Tratamiento aplicado | Cambiar la coma por un punto y convertir a número |
| Por qué (y riesgo) | No se pierde información; un texto raro pasaría a nulo sin avisar, por eso se cuentan |

   Hacen falta **6 filas o más**. Revisad las ocho familias de la plantilla (duplicados, tipos incorrectos, formatos de fecha, categorías inconsistentes, valores imposibles, *outliers*, nulos e incoherencias entre columnas) y marcad los tres defectos más graves. Si no llegáis a seis, mirad la «Ayuda 3» de la ficha del reto.

9. **Preguntas al cliente.** Lo que no podéis decidir solos va al apartado 5 del informe, con lo que haréis mientras tanto. Ejemplos de la plantilla: «¿por qué hay precios con coma?», «¿el 0 en precio significa "gratis" o "sin dato"?», «¿qué hace el sistema con el país cuando reserva una agencia?». Apuntad al menos una. Para repasar las técnicas, los cuadernos `S06_01` y `S06_02` hacen lo mismo con más explicación.

**Qué deberíais ver.** Con este CSV, estas salidas (los recuentos son los del fichero del curso):

- `(1545, 18)`: 1545 filas y 18 columnas.
- `resumen_calidad`: nulos en **cuatro columnas** (`adultos` 46, `regimen` 61, `pais_origen` 115 y `precio_noche` 84), y `object` en `fecha_reserva`, `fecha_llegada` y `precio_noche`. `id_reserva` tiene 1500 valores distintos en 1545 filas.
- `Filas duplicadas exactas: 45` y `Precios escritos con coma: 153`.
- `Fechas ISO: 1391 · dd/mm/aaaa: 154 · sin leer: 0`: dos formatos de fecha, y entre los dos se leen todas.
- `canal` con 9 valores (`web_intermediaria` 718, pero también `WEB_INTERMEDIARIA` 34, `Web Intermediaria` 33, `web intermediaria` 28, `Directo` 24, `directo` 16…) y `pais_origen` con 12 (`ES` 351 y también `España` 26 y `es` 23; `UK` 316 y también `Reino Unido` 22 y `uk` 23).
- Precios: mínimo `-50.0` y máximo `9999.0`, con `Precios <= 0: 4` y `precios > 1000: 4`; `Noches <= 0: 6`; `Antelaciones negativas: 5`.
- `Antelaciones que no cuadran con las fechas: 5`: cinco filas con `antelacion_dias` igual a `-3`.
- Un diagrama de caja con muchos puntos sueltos a la derecha. Un precio alto no es por sí solo un error: decidid si es imposible o solo raro.

Y en el informe, una tabla con **6 filas o más**. La plantilla da una columna para cada cosa: número, defecto, columna(s), filas afectadas, cómo lo detectasteis, causa probable, tratamiento aplicado y por qué (y riesgo).

**Hecho cuando:** la tabla tiene 6 o más defectos con sus datos completos y hay al menos una pregunta para el cliente.

## Si algo falla

| Veis… | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'crudo' is not defined` | La celda de carga no se ha ejecutado (o Colab se reinició). | Ejecutad desde arriba: la celda del paso 2 primero. |

| Veis… | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'iso' is not defined` | La celda de fechas (paso 4) no se ejecutó antes de la de incoherencias (paso 7). | Ejecutad la del paso 4 y luego la del 7. |
| `KeyError: 'precio_noches'` | El nombre de la columna no es exacto. | Escribid `crudo.columns` y copiad el nombre tal cual, con sus guiones bajos. |
| `HTTPError: HTTP Error 404: Not Found` al cargar | La dirección de `BASE_URL` o el nombre del CSV tienen una errata. | Copiad de nuevo la celda 2 y comprobad `reservas_sucio.csv`, todo en minúsculas. |
| `ValueError: time data "19/11/2024" doesn't match format "%Y-%m-%d"` | Se ha quitado `errors="coerce"` o se ha usado un solo formato para toda la columna. | Dejad las dos líneas `iso` y `espanol` con `errors="coerce"`: así cada una lee solo lo suyo. |
| Una celda se ejecuta pero no muestra nada | Colab enseña solo la **última** expresión; un resultado en medio de la celda no se ve. | Envolvedlo en `print(…)` o ponedlo como última línea. |
| La tabla del informe se ve rota en la vista previa de GitHub | Falta una `\|` en una fila o hay un salto de línea dentro de una celda. | Cada fila, en una sola línea y con el mismo número de `\|` que la cabecera. |

## Paso 3 · Función de limpieza y partición train/test (día 2, 15 minutos)

1. Escribir `limpiar_reservas(df)`: una función con docstring que aplica **todos** vuestros tratamientos.
2. Comprobar el antes y el después.
3. Separar *train* y *test*.

**En sencillo.** Una función de limpieza es como un programa de lavado: metéis la ropa sucia, lo ponéis en marcha y sale limpia, **siempre igual**, y la prenda original no se estropea porque se lava una copia. Eso es `limpiar_reservas`: recibe la tabla sucia y devuelve la limpia más un pequeño resumen de lo que ha cambiado. Después se separa una parte de las reservas, el *test*, y se guarda sin mirar, como el examen que el profesor no enseña antes de tiempo. No hace falta entender cada símbolo: copiad la función, ejecutadla y comprobad con los números que sale lo que esperáis. Lo que sí tenéis que decidir es qué columnas sirven para predecir y cuáles no.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Función (`def`) y `return`** | `def` crea un trozo de código con nombre; `return` es lo que devuelve al terminar. |
| **Docstring** | El texto entre triples comillas justo debajo de `def`: explica qué hace la función. |
| **Copia (`.copy()`)** | Una tabla igual pero independiente: lo que le hagáis no afecta a `crudo`. |
| **Diccionario (`{…}`)** | Una lista de parejas «nombre: valor». Aquí sirve de tabla de equivalencias (`"españa": "ES"`) y de resumen de cambios. |
| **Regla fija** | Una regla de limpieza que no depende de los datos: «el precio va entre 20 y 1000». |
| **`X` e `y`** | `X` son las columnas con las que se predice; `y` es lo que se quiere predecir (`cancelada`). |
| ***Train* y *test*** | *Train* (entrenamiento): reservas con las que el modelo aprende. *Test*: reservas apartadas que no se miran hasta el final. |
| **`stratify=y`** | Pide que *train* y *test* tengan la misma proporción de reservas canceladas. |
| **Fuga** | Usar para predecir algo que no se sabe a la hora de predecir. |

**Cómo se hace**

1. **Escribir la función.** Celda nueva. Es la función del mini-reto de `S06_02`, ya montada y con una línea más (`mes_llegada`). Si vuestra tabla del paso 2 tiene un defecto que aquí no aparece, añadid una línea antes del `return`.

```
EQUIV_PAIS = {"es": "ES", "españa": "ES", "uk": "UK", "reino unido": "UK", "de": "DE",
              "nl": "NL", "it": "IT", "fr": "FR", "nordicos": "Nordicos", "otros": "otros"}

def limpiar_reservas(tabla):
    """Limpia reservas_sucio.csv con reglas fijas y devuelve (tabla_limpia, resumen).

    No modifica la tabla que recibe: trabaja sobre una copia.
    Solo aplica reglas que no dependen de los datos (no calcula medias ni medianas).
    El resumen cuenta lo que ha cambiado: filas quitadas, valores imposibles pasados a nulo, etc.
    """
    t = tabla.copy()
    resumen = {"filas_antes": len(t)}

    t = t.drop_duplicates().reset_index(drop=True)
    resumen["duplicados_quitados"] = resumen["filas_antes"] - len(t)

    t["precio_noche"] = pd.to_numeric(t["precio_noche"].astype(str).str.replace(",", "."), errors="coerce")
    iso = pd.to_datetime(t["fecha_reserva"], format="%Y-%m-%d", errors="coerce")
    espanol = pd.to_datetime(t["fecha_reserva"], format="%d/%m/%Y", errors="coerce")
    t["fecha_reserva"] = iso.fillna(espanol)
    t["fecha_llegada"] = pd.to_datetime(t["fecha_llegada"], format="%Y-%m-%d")

    t["canal"] = t["canal"].str.strip().str.lower().str.replace(" ", "_")
    t["pais_origen"] = t["pais_origen"].str.strip().str.lower().map(EQUIV_PAIS)

    nulos_antes = t[["precio_noche", "noches"]].isna().sum().sum()
    t["precio_noche"] = t["precio_noche"].where(t["precio_noche"].between(20, 1000))
    t["noches"] = t["noches"].where(t["noches"].between(1, 60))
    resumen["imposibles_a_nulo"] = int(t[["precio_noche", "noches"]].isna().sum().sum() - nulos_antes)

    antelacion_vieja = t["antelacion_dias"]
    t["antelacion_dias"] = (t["fecha_llegada"] - t["fecha_reserva"]).dt.days
    resumen["antelaciones_recalculadas"] = int((t["antelacion_dias"] != antelacion_vieja).sum())

    t["mes_llegada"] = t["fecha_llegada"].dt.month
    resumen["filas_despues"] = len(t)
    return t, resumen
```

   Qué hace cada línea:

   - `EQUIV_PAIS = {…}`: tabla de equivalencias «cómo viene escrito (en minúsculas) → cómo debe quedar». Un país que no esté aquí acabaría como nulo.
   - `def limpiar_reservas(tabla):` y las líneas entre triples comillas: el nombre de la función y su docstring. `tabla` es el nombre que tiene la tabla **dentro** de la función; `crudo` no se llama así allí.
   - `t = tabla.copy()`: trabaja sobre una copia. `resumen = {…}` abre el diccionario donde se apunta cada cambio.
   - `drop_duplicates().reset_index(drop=True)`: quita las filas repetidas y renumera. La resta siguiente cuenta cuántas se quitaron.
   - `precio_noche …`: cambia la coma por un punto y convierte a número. Las cuatro líneas de `iso`, `espanol` y fechas leen las dos formas de escribir la fecha (como en el paso 2).
   - `.str.strip().str.lower().str.replace(" ", "_")`: quita espacios sobrantes, pasa a minúsculas y cambia el espacio por un guion bajo: `Web Intermediaria` pasa a `web_intermediaria`.
   - `.map(EQUIV_PAIS)`: sustituye cada país por su equivalencia.
   - `.where(…between(20, 1000))`: conserva el valor si está en el rango y, si no, pone un nulo. **No se borra la fila.** Los límites (precio 20–1000 €, noches 1–60) son los de `S06_02`.
   - `antelacion_dias`: se recalcula como días entre reserva y llegada. `mes_llegada` es un número de 1 a 12 sacado de la fecha de llegada.

- `return t, resumen`: devuelve **dos** cosas, por eso se recogen con dos nombres.

2. **Ejecutarla y leer el resumen.** Celda nueva:

```
limpia, resumen = limpiar_reservas(crudo)
print(resumen)
print(crudo.shape, limpia.shape)
```

Qué hace cada línea:

- `limpia, resumen = …`: guarda la tabla limpia en `limpia` y el diccionario en `resumen`. `crudo` sigue igual.
- `print(resumen)`: filas antes y después, duplicados, valores imposibles y antelaciones corregidas.
- `crudo.shape, limpia.shape`: la forma de cada tabla, para comprobar que `crudo` no cambió.

3. **Antes y después.** Dos celdas más. Es el apartado 3 del informe: si una corrección no se nota en estos números, no sirve.

```
antes_despues = pd.DataFrame({
    "antes": [len(crudo), crudo.duplicated().sum(), crudo["canal"].nunique(), crudo["pais_origen"].nunique()],
    "después": [len(limpia), limpia.duplicated().sum(), limpia["canal"].nunique(), limpia["pais_origen"].nunique()],
}, index=["filas", "duplicados", "valores distintos de canal", "valores distintos de país"])
antes_despues
```

```
nulos = pd.DataFrame({"antes": crudo.isna().sum(), "después": limpia[crudo.columns].isna().sum()})
nulos[nulos.sum(axis=1) > 0]
```

- `pd.DataFrame({…}, index=[…])`: crea una tabla con dos columnas (`antes`, `después`) y una fila por cosa que se cuenta.
- `.nunique()`: cuántos valores distintos tiene una columna.
- `limpia[crudo.columns]`: deja en `limpia` solo las 18 columnas originales (la nueva `mes_llegada` no estaba antes).
- `nulos[nulos.sum(axis=1) > 0]`: enseña solo las columnas que tienen algún nulo antes o después.

Y una comprobación de las fechas, por si el día y el mes se hubieran cambiado:

```
print(limpia["fecha_reserva"].min().date(), limpia["fecha_reserva"].max().date())
```

- `.min()` y `.max()`: la fecha más antigua y la más reciente; `.date()` quita las horas.

4. **Elegir las columnas y separar *train* y *test*.** Para cada columna, una pregunta: «¿se conoce **al reservar**?». Quedan fuera `cancelada` (es `y`), `id_reserva` (un número de identificación no predice) y las dos fechas, que ya están resumidas en `antelacion_dias` y `mes_llegada`. Pista de cuaderno: `S06_03` (partición). Celda nueva:

```
columnas_num = ["antelacion_dias", "noches", "adultos", "ninos", "precio_noche",
                "peticiones_especiales", "cliente_repetidor", "cancelaciones_previas", "mes_llegada"]
columnas_cat = ["tipo_habitacion", "regimen", "canal", "pais_origen", "tarifa", "deposito"]
X = limpia[columnas_num + columnas_cat]
y = limpia["cancelada"]

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEMILLA, stratify=y)
print(X_train.shape, X_test.shape)
print(round(y_train.mean(), 3), round(y_test.mean(), 3))
```

Qué hace cada línea:

- `columnas_num` y `columnas_cat`: las columnas de números y las de texto. Se separan porque el paso 4 las trata distinto.
- `X = limpia[…]` e `y = limpia["cancelada"]`: elige las columnas con las que se predice y la que se quiere predecir.
- `train_test_split(…)`: separa el 20 % para *test* (`test_size=0.2`). `random_state=SEMILLA` fija el sorteo y `stratify=y` mantiene la proporción de canceladas.
- `y_train.mean()`: como `cancelada` es 0 o 1, la media es la proporción de canceladas.

**Qué deberíais ver.** Aproximadamente estas cifras con el CSV del curso:

- `print(resumen)`: `{'filas_antes': 1545, 'duplicados_quitados': 45, 'imposibles_a_nulo': 14, 'antelaciones_recalculadas': 5, 'filas_despues': 1500}`.
- `(1545, 18) (1500, 19)`: 1545 filas antes y 1500 después; una columna más, `mes_llegada`.
- La tabla de antes y después: filas 1545 → 1500, duplicados 45 → 0, valores distintos de `canal` 9 → 4 y de país 12 → 8.
- Los nulos pasan, por ejemplo, de 0 a 6 en `noches` (los valores imposibles) y de 115 a 112 en `pais_origen`. Los huecos no se rellenan aquí: lo hace el *pipeline* del paso 4.
- `2024-01-01 2025-12-30`: las fechas de reserva abarcan dos años. Un año fuera de ese rango sería señal de día y mes cambiados.
- `(1200, 15) (300, 15)` y `0.471 0.47`: 1200 reservas para entrenar, 300 para *test*, y casi el mismo porcentaje de canceladas en las dos.

**Hecho cuando:** `limpiar_reservas(crudo)` devuelve la tabla limpia y el resumen sin cambiar `crudo`, y *train* y *test* están separados antes de calcular nada que se aprenda de los datos.

### Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'EQUIV_PAIS' is not defined` | La función se ejecutó sin la tabla de equivalencias. | Ejecutad las dos primeras líneas junto con la función, en la misma celda. |
| `AttributeError: 'tuple' object has no attribute 'shape'` | `limpia = limpiar_reservas(crudo)` guarda las dos cosas juntas. | Escribid `limpia, resumen = …`, con dos nombres. |
| `ValueError: not enough values to unpack (expected 3, got 2)` | Se recogen más nombres de los que devuelve la función. | La función devuelve dos cosas: dos nombres. |
| `KeyError: 'precio_noches'` | El nombre de la columna no es exacto. | Copiad el nombre de `crudo.columns`. |
| `ValueError: time data "19/11/2024" doesn't match format "%Y-%m-%d"` | Falta `errors="coerce"` en `iso` o en `espanol`. | Dejad las dos líneas como en la función. |
| Los países salen como nulos que antes tenían valor | Un país escrito de otra forma no está en `EQUIV_PAIS`. | Mirad `crudo["pais_origen"].value_counts()` y añadid la variante, en minúsculas. |
| `KeyError: "['mes_llegada'] not in index"` | `X` se ha sacado de `crudo`, que no tiene esa columna. | `X` sale de `limpia`: `X = limpia[columnas_num + columnas_cat]`. |

## Paso 4 · Pipeline y baseline (día 2, 15 minutos)

1. Construir un `Pipeline` que impute, escale y codifique.
2. Añadir un modelo de referencia y un `DummyClassifier`.
3. Compararlos con validación cruzada y escribir la frase del resultado.

**En sencillo.** Tras la limpieza todavía quedan huecos, números en escalas muy distintas (una antelación de 300 días y un 1 de «cliente repetidor») y texto que el ordenador no sabe usar. Un *pipeline* es una **cadena de montaje**: la tabla entra por un lado, pasa por varias máquinas (rellenar huecos, igualar escalas, convertir texto en números) y sale al modelo, y la cadena entera se monta una vez y se reutiliza igual. Para saber si vuestro modelo vale algo, lo comparáis con un «modelo tonto» que no mira los datos y apunta siempre a lo más frecuente, y lo hacéis con *validación cruzada*: repetir la prueba cinco veces con trozos distintos. No hace falta entender por dentro la regresión logística: basta con que sepáis leer si gana al modelo tonto.

### Palabras nuevas de este paso

| Palabra | Qué significa, en sencillo |
|---|---|
| ***Pipeline*** | Una cadena de pasos (preparar datos y modelo) que se ejecuta siempre en el mismo orden. |
| `ColumnTransformer` | La parte de la cadena que prepara cada grupo de columnas de forma distinta: números por un lado, texto por otro. |
| **Imputar** (`SimpleImputer`) | Rellenar huecos: con la mediana en las columnas numéricas y con el valor más frecuente en las de texto. |
| **Escalar** (`StandardScaler`) | Poner las columnas numéricas en una escala parecida, para que una no pese más solo por tener números más grandes. |
| **Codificar** (`OneHotEncoder`) | Convertir una columna de texto en varias columnas de 0 y 1, una por cada valor posible. |
| ***Baseline* o modelo tonto** | El modelo de referencia, que no aprende nada y sirve de suelo: si el vuestro no lo supera, no aporta. |
| **Validación cruzada** | Trocear *train* en 5 partes y repetir 5 veces «entrenar con 4 y puntuar con la que sobra». |
| ***Fold*** | Cada una de esas 5 repeticiones; salen 5 puntuaciones. |
| **AUC (`roc_auc`)** | Una nota de 0,5 a 1: 0,5 es azar y 1 es perfecto. Mide si el modelo pone delante las reservas que se cancelan. |
| **Fuga** | Que el *test* (o información de fuera de lo que se está entrenando) influya en lo que el modelo aprende. |

### Cómo se hace

1. **Montar el preprocesado.** Celda nueva. Usa `columnas_num` y `columnas_cat` del paso 3:

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocesado = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")),
                      ("esc", StandardScaler())]), columnas_num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                      ("oh", OneHotEncoder(handle_unknown="ignore"))]), columnas_cat),
])
```

   Qué hace cada línea:

   - `from … import …`: trae las piezas de scikit-learn que se van a usar.
   - `ColumnTransformer([…])`: abre la lista de «ventanillas». Cada una tiene un nombre (`"num"`, `"cat"`), qué hace y a qué columnas se aplica.
   - `("num", Pipeline([…]), columnas_num)`: las columnas numéricas pasan primero por `SimpleImputer(strategy="median")` (huecos con la mediana) y luego por `StandardScaler()` (misma escala).
   - `("cat", Pipeline([…]), columnas_cat)`: las de texto pasan por `SimpleImputer(strategy="most_frequent")` (huecos con el valor más frecuente, también en `regimen`) y por `OneHotEncoder(handle_unknown="ignore")`, que no da error si en *test* aparece un valor nuevo.
   - Los nombres `"imp"`, `"esc"`, `"oh"`, `"num"` y `"cat"` los podéis cambiar, pero conviene dejarlos.

2. **Montar el modelo y el modelo tonto.** Celda nueva, con la **misma** cadena de preparación:

```
from sklearn.linear_model import LogisticRegression
from sklearn.dummy import DummyClassifier

modelo = Pipeline([("prep", preprocesado), ("clf", LogisticRegression(max_iter=1000))])
tonto = Pipeline([("prep", preprocesado), ("clf", DummyClassifier(strategy="prior"))])

matriz = preprocesado.fit_transform(X_train)
print(X_train.shape, "→", matriz.shape)
```

   Qué hace cada línea:

   - `modelo = Pipeline([("prep", …), ("clf", …)])`: la cadena completa: primero el preprocesado y al final la regresión logística, que predice si una reserva se cancela. `max_iter=1000` le da margen para ajustarse.
   - `tonto = …`: la misma cadena, pero el último paso es `DummyClassifier(strategy="prior")`, que solo mira la proporción de canceladas.
   - `preprocesado.fit_transform(X_train)`: aplica la preparación solo a *train* (es solo para mirar el resultado).
   - `print(X_train.shape, "→", matriz.shape)`: tamaño antes y después. Las columnas de texto se han abierto en muchas de 0 y 1.

3. **Validación cruzada.** Celda nueva:

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEMILLA)
auc_tonto = cross_val_score(tonto, X_train, y_train, cv=cv, scoring="roc_auc")
auc_modelo = cross_val_score(modelo, X_train, y_train, cv=cv, scoring="roc_auc")

resultados = pd.DataFrame({
    "media": [auc_tonto.mean(), auc_modelo.mean()],
    "desviación": [auc_tonto.std(), auc_modelo.std()],
}, index=["modelo tonto", "regresión logística"]).round(3)
resultados
```

   Qué hace cada línea:

   - `StratifiedKFold(n_splits=5, shuffle=True, random_state=SEMILLA)`: define los 5 trozos, mezclados con una semilla y con la misma proporción de canceladas en cada uno.
   - `cross_val_score(modelo, X_train, y_train, cv=cv, scoring="roc_auc")`: en cada vuelta entrena con 4 trozos y puntúa con el que sobra. Devuelve 5 notas.
   - Se usa `X_train`, nunca `X_test`. Como el imputador y el escalador están **dentro** de la cadena, en cada vuelta aprenden solo de los 4 trozos de entrenamiento: eso evita la fuga.
   - `.mean()` y `.std()`: la media de las 5 notas y cuánto varían (desviación).
   - `scoring="roc_auc"` es la métrica del paso. Si preferís `"f1"` (equilibra aciertos y falsas alarmas entre las canceladas), cambiad solo esa palabra y decid cuál elegís y por qué.

4. **Escribir la frase.** Va en el apartado 4 del informe, con vuestras cifras. Ejemplo con las de este fichero: «Nuestro modelo obtiene un AUC medio de 0,758 ± 0,025 frente a 0,500 del modelo tonto. Eso significa que ordena las reservas mucho mejor que el azar». Con `"f1"`: «Un F1 medio de 0,660 ± 0,029 frente a 0,000 del modelo tonto». Para el nivel extra, mirad la partición temporal en `S06_04` y la fuga en `S06_03`.

**Qué deberíais ver.** Aproximadamente estas cifras con el CSV del curso:

- `(1200, 15) → (1200, 34)`: las 15 columnas pasan a 34.
- Una tabla de dos filas: modelo tonto con media `0.500` y desviación `0.000`; regresión logística con media `0.758` y desviación `0.025`.
- Las cinco notas de `auc_modelo` van de 0,73 a 0,81, y las del tonto valen 0,5 las cinco veces. Si lo cambiáis a `"f1"`, el tonto da 0,0 y vuestro modelo unos 0,66.
- Vuestro modelo **por encima del tonto**. Si no, algo falla: mirad la «Ayuda 2» de la ficha del reto.

**Hecho cuando:** el cuaderno tiene el *pipeline*, la tabla de resultados y la frase con media ± desviación, y sabéis explicar por qué el escalado y la imputación no usan el *test*.

### Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'X_train' is not defined` | No se ejecutó la celda del paso 3 con la partición. | Ejecutad desde arriba o al menos hasta el paso 3. |
| `ValueError: Input X contains NaN.` y `LogisticRegression does not accept missing values encoded as NaN natively` | Falta un `SimpleImputer` en alguna de las ramas, o el modelo recibe columnas que no pasaron por el preprocesado. | Comprobad que `columnas_num` y `columnas_cat` están completas y que cada rama lleva su imputador. |
| `ValueError: Some column names are not columns of the dataframe: {'precio'}` | Un nombre de `columnas_num` o `columnas_cat` no existe en `X`. | Comparad con `X_train.columns` y corregid la errata. |
| `ValueError: Cannot use median strategy with non-numeric data` y `could not convert string to float: 'desayuno'` | Una columna de texto (por ejemplo `regimen`) está en la rama numérica. | Pasadla a `columnas_cat`. |
| `NameError: name 'preprocesado' is not defined` | La celda 1 no se ejecutó. | Ejecutadla antes de la celda del modelo. |
| El modelo tonto da exactamente 0,500 con desviación 0,000 | Es lo esperado: siempre apunta a lo mismo. | No es un error; es el suelo con el que se compara. |

## Paso 5 · Informe, README y revisión con la rúbrica (día 2, 20 minutos)

1. Completar `informe_calidad.md` a partir de la plantilla.
2. Rellenar el `README.md` y comprobar *Reiniciar y ejecutar todo*.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado.

**En sencillo.** Ya tenéis el trabajo hecho; falta **contarlo** para que otra persona lo entienda sin vosotros delante. El informe es para el gerente del hotel, que no programa: qué estaba mal, cuántas filas afecta y qué habéis hecho con cada cosa. El README es el «manual de instrucciones»: con él, alguien de fuera debería poder abrir el cuaderno y obtener las mismas cifras. Después, una persona que no ha escrito esa parte lo revisa con la rúbrica, como un corrector con ojos frescos. Casi todo sale de lo que ya habéis ejecutado en los pasos 2, 3 y 4; solo hay una celda de código nueva y es pequeña.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Informe** | El documento (`informe_calidad.md`) donde se cuenta el trabajo a quien no programa. |
| **README** | La «portada» de la carpeta: qué hay, cómo ejecutarlo y quién hizo qué. |
| **Reiniciar y ejecutar todo** | Borra la memoria del cuaderno y ejecuta todas las celdas de arriba abajo. Es la prueba de que no depende de nada «escondido». |
| **Versión** | El número de edición de Python o de una librería. Con otra versión, algunos números pueden cambiar un poco. |
| **Rúbrica** | La tabla con la que se evalúa: cuatro criterios, cada uno de 1 a 4. |
| **Sesgo / discriminar** | Que el modelo trate peor a un grupo (por ejemplo, por su país) sin que haya una razón válida. |
| ***Commit*** | Guardar vuestros cambios en GitHub con un mensaje corto y vuestro nombre. Sin pulsar **Commit changes**, no se guarda nada. |
| ***Review* y *retro*** | La *review* enseña el trabajo a otros equipos; la *retro* es hablar de cómo trabajasteis. |

**Cómo se hace**

1. **Rellenar el informe.** Abrid `informe_calidad.md` (lo creasteis en el paso 2; para editarlo, pulsad el icono del lápiz) y completad cada apartado con **vuestras cifras**, sacadas de las celdas ya ejecutadas:

| Apartado | Qué se escribe | De dónde sale |
|---|---|---|
| 1 · Resumen para el gerente (máx. 5 líneas) | ¿Se puede usar el fichero tal cual? ¿Qué fracción de filas necesita corrección? ¿Qué es lo más grave? | Pasos 2 y 3 |
| 2 · Tabla de defectos | 6 o más filas, con los **tres más graves en negrita** | Paso 2 |
| 3 · Antes y después | Una tabla pequeña: filas 1545 → 1500, duplicados 45 → 0, valores de `canal` 9 → 4 y de país 12 → 8, y los nulos que cambian | Paso 3 |
| 4 · *Pipeline* y modelo de referencia | Columnas numéricas y de texto; qué va **dentro** del *pipeline* (imputar, escalar, codificar) y qué **antes** (las reglas fijas de `limpiar_reservas`); la frase con media ± desviación | Pasos 3 y 4 |
| 5 · Preguntas para el cliente | Al menos 3, con lo que haréis mientras no respondan | Paso 2 |
| 6 · IA responsable | Ejemplos del dataset (ver el punto 2) | Punto 2 |
| 7 · Cómo reproducirlo | Versiones de Python y librerías, orden de ejecución, ficheros | Punto 3 |

   **Para el nivel mínimo** bastan los apartados 1, 2 y 3. **Para el estándar**, completad también el resto. La tabla del apartado 3 se escribe a mano con los números que salieron en el paso 3, por ejemplo:

```
| | Antes | Después |
|---|---|---|
| Filas | 1545 | 1500 |
| Duplicados | 45 | 0 |
| Valores distintos de `canal` | 9 | 4 |
```

2. **Responder a la IA responsable con datos.** Las preguntas son: ¿qué pasa si el país de origen se imputa mal? ¿Podría el modelo discriminar por nacionalidad? ¿Hay columnas que no deberían usarse? Para contestar con números, celda nueva:

```
print(limpia.groupby("pais_origen")["cancelada"].agg(["mean", "count"]).round(2))
print(limpia["pais_origen"].isna().sum(), round(limpia["pais_origen"].isna().mean(), 3))
```

   Qué hace cada línea:

   - `limpia.groupby("pais_origen")`: agrupa las reservas por país.
   - `["cancelada"].agg(["mean", "count"])`: para cada grupo, la proporción de canceladas (`mean`) y cuántas reservas hay (`count`).
   - `.isna().sum()` y `.isna().mean()`: cuántas reservas no tienen país y qué parte del total son.

   Con estas cifras podéis escribir, por ejemplo: «Italia cancela mucho más (0,60 sobre 77 reservas) que Países Bajos (0,37 sobre 75). Faltan 112 países (7,5 %); el imputador pondría `ES`, el más frecuente, y mezclaría reservas de otros países. Si el modelo usa el país, podría tratar distinto a quien reserva desde un lugar concreto». Decid también qué columnas dejaríais fuera (por ejemplo `id_reserva`) y por qué.

3. **Escribir el README.** En la carpeta `sprint-06/` del repositorio, abrid `README.md` y pulsad el icono del lápiz. Cuatro cosas, en pocas líneas: qué hace el cuaderno; cómo ejecutarlo y en qué orden (*abrir en Colab y ejecutar de arriba abajo*); las versiones; y quién hizo qué y **qué le pedisteis a la IA y qué revisasteis**. Las versiones salen de esta celda, que ejecutáis en el cuaderno:

   ```
   import sys, pandas, sklearn
   print(sys.version, pandas.__version__, sklearn.__version__)
   ```

   Qué hace cada línea:

   - `import sys, pandas, sklearn`: trae los tres módulos para preguntarles su versión.
   - `print(sys.version, pandas.__version__, sklearn.__version__)` : escribe la versión de Python, la de pandas y la de scikit-learn. Copiadlas tal cual a vuestro README. Salen otras en cada ordenador, y eso es normal.

4. **Reiniciar y ejecutar todo.** En Colab, *Entorno de ejecución → Reiniciar y ejecutar todo* y confirmad. Mirad hasta el final que ninguna celda muestre un error en rojo. Si falla, hay celdas que dependen de un orden: arreglad antes de seguir y repetid.
5. **La revisión con la rúbrica (5 minutos).** La hace una persona que **no escribió** esa parte. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: ejecutar el cuaderno según el README, leer el informe como el gerente, mirar el código (¿algún `fit` usa el *test*?), poner un nivel y apuntar el resultado en la sección «Revisión con la rúbrica» del final del informe. Es una tabla con un nivel de 1 a 4 por criterio y **una frase de prueba** (lo que ha comprobado), más «Lo primero que mejoraríamos». Si hay tiempo, arreglad lo más fácil; lo demás queda como mejora. Si sobran minutos, empezad el extra.

**Qué deberíais ver.** `informe_calidad.md` con todos los apartados rellenos y la sección «Revisión con la rúbrica» completa, un `README.md` con los comandos y las versiones, y el cuaderno ejecutándose de arriba abajo sin errores. La celda del punto 2 escribe, aproximadamente, `IT 0.60 77`, `NL 0.37 75`, `ES 0.45 388` y, en la segunda línea, `112 0.075`. La de las versiones escribe una línea con Python, pandas y scikit-learn (en Colab serán las suyas).

**Hecho cuando:** otra persona podría repetir el cuaderno siguiendo solo el README, y el informe tiene anotados el nivel y la prueba de cada criterio.

**Después, 35 minutos de entrega, review y retro** (también el día 2):

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) en Colab, *Archivo → Descargar → Descargar .ipynb* (después de *Reiniciar y ejecutar todo*); (2) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-06`**; (3) *Add file → Upload files* y arrastrad el fichero; (4) escribid un mensaje corto y pulsad **Commit changes**. Los nombres, **en minúsculas y sin espacios**; el CSV va a `data/` solo si aún no está. Un reparto posible: la persona A sube `informe_calidad.md` («Añado informe de calidad»), la B sube `reto_S06.ipynb` («Añado cuaderno»), la C actualiza `README.md` («Completo el README»).
- **La autoevaluación.** Cada persona copia la plantilla (botón **Raw**), la rellena y la sube a `sprint-06/` como `autoevaluacion_nombre.md`; la coevaluación va dentro del mismo fichero.
- **La review (5 minutos por equipo).** Enseñad: (1) la tabla de defectos, los 3 más graves y por qué; (2) el cuaderno corriendo desde cero; (3) la métrica del *baseline* frente al modelo tonto.
- **La retro.** Pensad: ¿qué defecto nos costó más encontrar? ¿Qué decisión de limpieza discutimos más? ¿Qué haríamos distinto con el siguiente fichero del cliente?

**Qué deberíais ver.** En la carpeta `sprint-06` del repositorio, los tres ficheros del equipo y una hoja de autoevaluación por persona (**3 + N**), con *commits* de personas distintas en el historial (el icono del reloj).

## Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| Al *Reiniciar y ejecutar todo* sale un error y se para | Una celda depende de otra que está más abajo o que falla. | Leed el error de la **primera** celda en rojo, corregidla y repetid. |

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'limpia' is not defined` | La celda de la limpieza no se ha ejecutado antes de esta. | Ejecutad desde arriba con *Reiniciar y ejecutar todo*. |
| `KeyError: 'pais_origen '` | El nombre de la columna lleva un espacio o una errata. | Copiadlo de `limpia.columns`. |
| El archivo aparece fuera de `sprint-06/` | Se subió desde la raíz del repositorio. | Entrad primero en `sprint-06` y repetid la subida; el de la raíz se borra desde el lápiz o los tres puntos del archivo. |
| La tabla del informe se ve rota en *Preview* | Falta una `\|` en alguna fila o hay un salto de línea dentro de una celda. | Cada fila, en una sola línea y con las mismas `\|` que la cabecera. |
| No sabéis qué nivel poner en la rúbrica | Los niveles están descritos en entrega y rúbrica. | Leed la fila del criterio, elegid el nivel cuyo texto se cumple **entero** y escribid la prueba. |

## Ayuda 1 · Cómo usar la IA en este reto

La limpieza de datos es un terreno donde la IA ayuda mucho, pero **donde también introduce errores sutiles**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Solo librerías del curso.** No instaléis paquetes extra que os «arreglen» los datos automáticamente.
- **Toda corrección va en el código**, no en el CSV ni en un clic de Excel.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA (los datos de este reto son sintéticos, pero se coge el hábito).

| Para qué | Ejemplo de petición | Qué comprobáis |
|---|---|---|
| Entender una función de pandas | «Explícame `fillna`, `drop_duplicates` y `to_datetime` con un ejemplo muy pequeño» | Que probáis cada ejemplo y lo entendéis |
| Depurar un error | «Este código da `KeyError: 'precio'`. El DataFrame tiene estas columnas: *[lista]*. ¿Qué puede pasar?» | Que arregláis la causa, no solo el síntoma |
| Revisar la fuga | «Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test para aprender?» | Que entendéis **por qué** hay o no fuga |
| Redactar el informe | «Reescribe esta tabla de defectos de forma más clara» | Que **no cambie cifras ni defectos** |

## Ayuda 2 · Errores frecuentes (y cómo evitarlos)

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| Calcular la mediana con **todos** los datos antes de partir | «Total, es solo la mediana» | Partid primero; imputación y escalado se ajustan **solo con *train***, dentro del `Pipeline` |
| Borrar filas «raras» sin avisar | Se quieren datos limpios | Cada fila eliminada se **cuenta y se justifica** en el informe |
| Informe que dice «hay nulos» sin más | Falta el análisis | Para cada defecto: dónde, cuántas filas, causa, tratamiento y razón |
| Un modelo que «acierta mucho» y se celebra | La exactitud engaña si una clase pesa mucho más | Comparadlo con `DummyClassifier` y mirad F1 o ROC-AUC |
| El cuaderno solo funciona con las celdas en cierto orden | Se editaron celdas sueltas | *Reiniciar y ejecutar todo* antes de entregar |

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| Convertir las fechas y que salgan nulas o con día y mes cambiados | Hay dos formatos mezclados y se aplica solo uno | Probar cada formato por separado con `errors="coerce"` y comprobar el año mínimo y máximo |
| Usar columnas que delatan la respuesta o que no se conocen al reservar | Parece que ayudan | Preguntad por cada columna: ¿se conoce **al reservar**? |
| Que la limpieza cambie el CSV original o la tabla `crudo` | Se modifica la tabla en lugar de una copia | Empezar la función con `.copy()` y no escribir nunca en `crudo` |

## Ayuda 3 · Si os atascáis

1. **No encontramos más defectos:** probad `df.describe()`, `df.isna().sum()`, `df.duplicated().sum()` y `df[columna].value_counts()` en cada columna, y revisad las ocho categorías de la tabla del paso 2.
2. **No sabemos si algo es un error o un dato real:** apuntadlo como **pregunta para el cliente** y explicad qué hacéis mientras tanto.
3. **Una función de pandas no hace lo que queremos:** probadla con un ejemplo de 3 filas inventadas, y repasad el cuaderno correspondiente y su código explicado. Después, preguntad al equipo y al docente.